🎬 AI-Powered Movie Recommendation System

Project Description

This project is a **Console Application** built with **C# and .NET 8** that provides personalized movie recommendations using Artificial Intelligence.

The system allows users to:

- Register and login to their accounts
- Browse a collection of 50 movies
- Search movies by title, genre, year, director, or rating
- Rate movies from 1 to 5 stars
- Remove their ratings
- View their watch history
- Receive AI-powered movie recommendations based on their preferences
- Get trending movies, recently watched, and export recommendations to file

OOP Concepts Used

1. Encapsulation

We used private fields with public properties in all model classes (User, Movie, Rating). Validation logic is applied in property setters to ensure data integrity.

Example from `User.cs`:

```csharp private string _password; public string Password { get { return _password; } set { _password = value; } }
```

2. Inheritance

We created a hierarchy where `Person` is an abstract base class, `User` inherits from it, and `Admin` inherits from `User`.

Person (Abstract)

    ├── User

        ├── Admin

Example:

```
public abstract class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
}

public class User : Person
{
    public string Username { get; set; }
    public string Password { get; set; }
}
```

3. Abstraction

The `Person` class is abstract, and we created abstract methods that must be implemented by derived classes.

4. Interfaces

We created three interfaces to enforce contracts across our classes:

Interface	Purpose
`IRecommendable`	For recommendation engines
`ISearchable<T>`	For search functionality
`IDataStorage<T>`	For data persistence

Example:

```
public interface ISearchable<T>
{
    List<T> SearchByTitle(string title);
    List<T> SearchByGenre(int genreId);
    List<T> SearchByYear(int year);
}
```

5. Polymorphism

We implemented two different recommendation strategies that both use the same interface:

- `ContentBasedFiltering` - Recommends based on movie genres and features
- `CollaborativeFiltering` - Recommends based on similar users' preferences

Both classes implement the same `GetRecommendations()` method but behave differently.

AI Logic Explanation

Cosine Similarity Formula

We use Cosine Similarity to calculate how similar two users are based on their movie ratings

Similarity(A,B) = (A · B) / (||A|| × ||B||)

Where:

- `A · B` is the dot product of ratings from users A and B
- `||A||` and `||B||` are the magnitudes of the rating vectors

Strategy 1: Content-Based Filtering

This strategy recommends movies based on the content the user already likes:

Factor	Weight	Description
Genre Matching	50%	Compares user's favorite genres with movie genres
Director Similarity	20%	Checks if user watched other movies by same director
Movie Popularity	30%	Uses the movie's average rating

Example: If a user rated "Inception" (Action, Sci-Fi, Thriller) highly, the system will recommend other movies with similar genres.

Strategy 2: Collaborative Filtering

This strategy finds users with similar rating patterns:

1. Calculate similarity between current user and all other users using Cosine Similarity
2. Find the top 10 most similar users
3. Recommend movies that similar users liked but the current user hasn't seen yet
4. Weight recommendations by the similarity score

Strategy 3: Hybrid Approach

We combine both strategies for better recommendations:

Final Score = (Content-Based Score × 0.4) + (Collaborative Score × 0.6)

- 40% weight to Content-Based Filtering
- 60% weight to Collaborative Filtering

Dynamic Learning

Every time a user rates a new movie:

1. The system updates the user's profile
2. The recommendation engine recalculates recommendations
3. The movie's average rating is updated

System Architecture

```
MovieRecommendationSystem/
│
├── .gitignore                    # Git ignore file
├── DOCUMENTATION.md              # Project documentation
├── MovieRecommendationSystem.sln      # Solution file
│
└── MovieRecommendationSystem.Console/  # Main project
    │
    ├── Models/                   # Data structures
    │   ├── Person.cs             # Abstract base class
    │   ├── User.cs               # User model
    │   ├── Movie.cs              # Movie model
    │   ├── Rating.cs             # Rating model
    │   └── Genre.cs              # Genre model
    │
    ├── Services/                 # Business logic
    │   ├── DataStorageService.cs      # JSON read/write
    │   ├── AuthService.cs             # Login/Register
    │   ├── MovieService.cs            # Movie operations
    │   └── RatingService.cs           # Rating operations
    │
    ├── Recommendation/           # AI algorithms
    │   ├── SimilarityCalculator.cs    # Cosine Similarity
    │   ├── ContentBasedFiltering.cs   # Content-based strategy
    │   ├── CollaborativeFiltering.cs  # Collaborative strategy
```

```
│   └── RecommendationEngine.cs     # Hybrid engine
│
├── Interfaces/                 # Contracts
│   ├── IRecommendable.cs
│   ├── ISearchable.cs
│   └── IDataStorage.cs
│
├── UI/                         # User interface
│   └── ConsoleHelper.cs          # Menus, colors, tables
│
├── Data/                       # JSON storage
│   ├── users.json              # User data
│   ├── movies.json             # Movie data (50 movies)
│   ├── ratings.json            # Rating data (100+ ratings)
│   └── genres.json             # Genre data (8 genres)
│
└── Program.cs                  # Main entry point
```

Bonus Features Implemented

Feature	Description
🔥 Trending Movies	Shows top 5 highest-rated movies
🕐 Recently Watched	Shows user's 5 most recent ratings
📊 AI Confidence Score	Shows percentage match for each recommendation
💾 Save Recommendations	Exports recommendations to a .txt file

Challenges Faced and Solutions

Challenge 1: Git Merge Conflicts

Problem: Both team members worked on different branches, and when merging, conflicts appeared.

Solution: We divided the work clearly:

- Ahmed worked on Models + Services + Recommendation
- Sarah worked on UI + Program.cs
- Used Pull Requests on GitHub to merge safely

Challenge 2: JSON File Path Issues

Problem: The program couldn't find the JSON files when running from different locations.

Solution: We used relative paths with `Directory.GetCurrentDirectory()` instead of hardcoded paths.

Challenge 3: Interface Implementation Errors

Problem: The `RecommendationEngine` class didn't implement the `IRecommendable` interface correctly.

Solution: We added the required method:

```
public List<(Movie movie, double score)> GetRecommendations(User user, int topN = 5)
{
    return GetHybridRecommendations(user, topN);
}
```

Challenge 4: Git index.lock Error

Problem: Git kept showing "Unable to create index.lock" when switching branches.

Solution: We deleted the lock file manually:

```
del .git\index.lock
```

Challenge 5: Sample Movies Still Showing

Problem: The program showed "Sample Movie" instead of real movies.

Solution: We deleted the `Data` folder and let the program regenerate it from the updated `MovieService.cs` that contains 50 real movies.

How to Run the Project

Prerequisites

- .NET 8 SDK installed
- Git (optional, for cloning)

Steps

1. **Clone the repository:**

```
git clone https://github.com/nouraalmaskari11-wq/MovieRecommendationSystem.git
cd MovieRecommendationSystem
```

2. **Restore dependencies:**

```
cd MovieRecommendationSystem.Console
dotnet restore
```

3. **Run the application:**

```
dotnet run
```

Sample Login

Role	Username	Password
Admin	`admin`	`admin123`
User	`user2` to `user10`	`pass123`

Features Demo

Menu Option	Description
1	Browse all 50 movies
2	Search by title, genre, year, director, or rating
3	Rate a movie (1-5 stars)
4	Get AI-powered recommendations with confidence score
5	View your rating history
6	Remove a rating
7	Logout
8	View trending movies (top-rated)
9	View recently watched movies
10	Save recommendations to a .txt file

Team Members

Name	Role	Responsibilities
Noura	Backend Developer	Models, Services, Recommendation Engine, AI Logic
Elham	Frontend Developer	UI, ConsoleHelper, Program.cs, Integration

Technologies Used

Technology	Purpose
C#	Programming language
.NET 8	Framework
LINQ	Data manipulation and queries
Newtonsoft.Json	JSON serialization/deserialization
ML.NET	AI concepts implementation
Git & GitHub	Version control and collaboration

Project Statistics

Category	Count
Movies	50 real movies
Users	10 users (1 admin + 9 regular)
Ratings	100+ sample ratings
C# Files	17 classes
Lines of Code	~2000 lines
GitHub Commits	40+ commits
Branches	3 (main, feature/backend-ai, feature/ui-integration)

Conclusion

This project successfully implements an **AI-powered movie recommendation system** using:

- Full OOP principles (Encapsulation, Inheritance, Abstraction, Interfaces, Polymorphism)
- Two AI recommendation strategies (Content-Based + Collaborative)
- Cosine Similarity algorithm
- Hybrid approach (40% / 60% weighting)
- JSON data persistence
- LINQ for data queries
- Clean, modular architecture
- 4 bonus features

GitHub Repository

https://github.com/nouraalmaskari11-wq/MovieRecommendationSystem

Documentation prepared by: Noura& Elham
Date: May 2026